Replacing raw bits with token logits

TinyGSM → GSM8K. How the V-way token-softmax head was built, how the Isambard run behaved, and whether it beat the per-bit head. Written 2 Oct 2026 from results on disk; no GPU was used.

> **Bottom line: worse, at every point we measured.** With everything else matched, the token head reached about **0.55× the bits accuracy**: 1.6% vs 3.0% at 100k steps and 4.1% vs 7.2% at 200k (significant on every seed). An independent pair on CSD3 at 500k steps agrees (2.6% vs 4.9%). Training was stable, but the run was stopped at 414k steps for budget and its later checkpoints were never evaluated. The failure is execution: **93–97% of token programs do not run**, although the ones that do run are right more often than bits programs on the same problems.

1. What we changed

CoBit normally writes each token as a 16-bit codeword and predicts each bit on its own. The idea was to predict the whole token at once: one probability over the 49,153-word vocabulary per position, so the model can express which *tokens* are likely instead of 16 independent coin flips (which can only describe a product distribution over codewords).

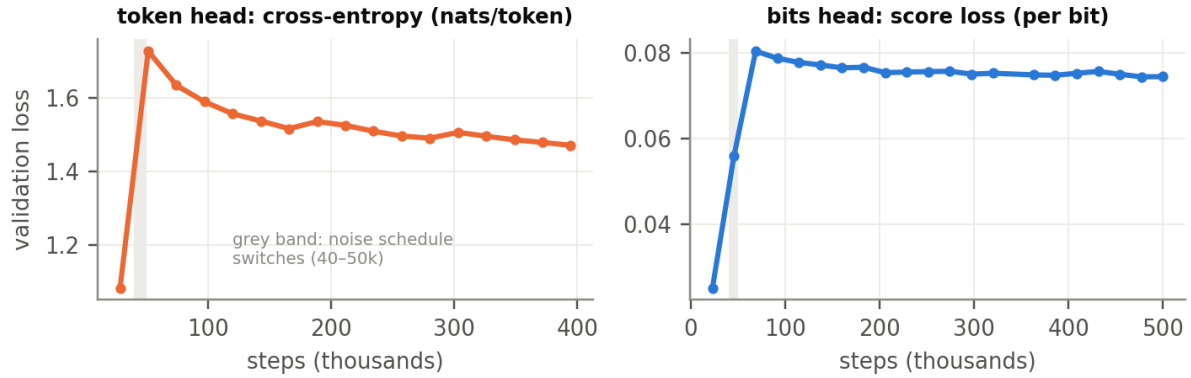
	Bits head (production)	**Token head (this study)**
State per position	16 bits in {0,1} (8,192 numbers per sequence)	one-hot vector over V = 49,153 tokens (512 positions)
Output	1 logit per bit → sigmoid	V logits per token → softmax
Matched filter	added per bit: $(x_t - ½)/σ²$	same term per vocabulary entry (the exact Gaussian likelihood for a one-hot code)
Loss	score loss on bits (binary_sm)	cross-entropy on the token (token_ce), same EDM weight
Decode	threshold bits → codeword → token	argmax over the vocabulary
Parameters	133.8M	212.3M (the V-way input and output layers)

Held the same: the 12-layer, 768-wide transformer, the Gaussian noise process and σ range, the learned noise schedule, self-conditioning, CFG dropout (p=0.1), EMA, AdamW at a constant lr 1e-4 with 2.5k warmup, effective batch 512, seed 42, 500k-step target, the same data and 512-token canvas. The bits control is `ord_gh500k_none_s42`, trained in the same week on the same cluster.

Not held the same (worth knowing when reading the result): the loss family changes with the head (cross-entropy vs score loss), the token logits come out of a narrow 128-wide head layer (a wider `token_full` head exists but was not used), and the token run used 8 GPUs (2 nodes) vs 4 for bits, because the V-way logits need much more memory.

2. How the Isambard run behaved

	Token run (tok_gh500k_token_ce_s42)	Bits control (ord_gh500k_none_s42)
Hardware	8 × GH200 on 2 nodes (first 5k steps on 1 node)	4 × GH200 on 1 node
Speed	2.66 optimiser steps/s	≈ 3.95 steps/s
Steps reached	**414,000 of 500,000**, stopped 26 Sep at the budget line	500,000 (finished 25 Sep)
Stability	no NaN/Inf, divergence guard never fired	no divergence
Interruptions	3 NCCL hangs at epoch boundaries (5-epoch entropy-plot callback, audit T2); each resumed from the epoch checkpoint, no steps lost	none
σ_{data} (rank-0 estimate)	0.4575	0.3998
Approx. GPU-h (from log times)	≈ 355	≈ 141

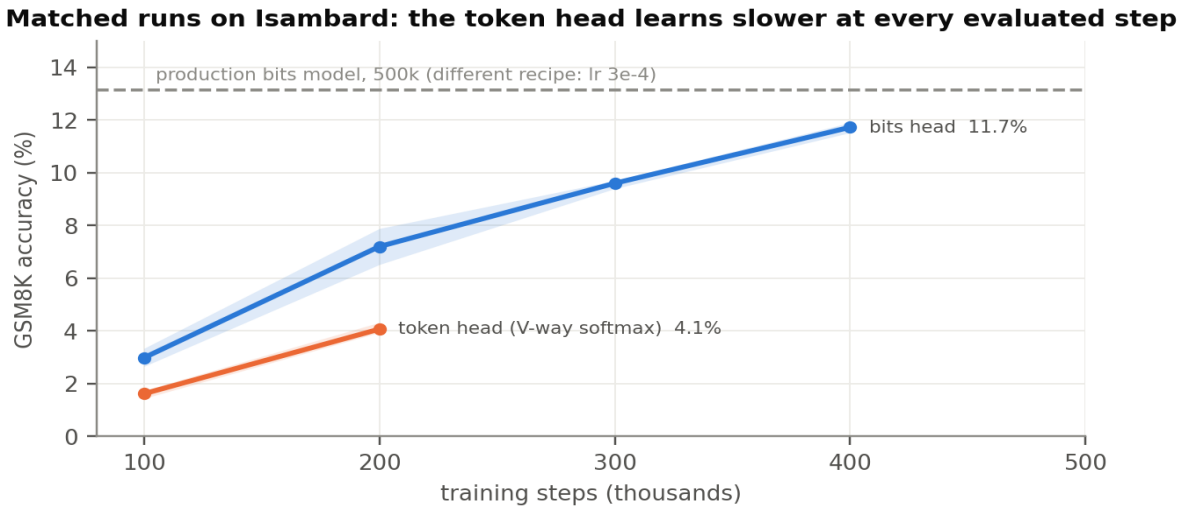


Validation loss per epoch, from the runs' TensorBoard logs. The two losses are different quantities and are not comparable across panels. Both jump when the noise schedule switches to the learned distribution (40–50k), then improve slowly and steadily: no instability.

3. Results

Every cell: full GSM8K test set (1319 problems), 3 sampling seeds, Karras/DDIM 256 steps, deterministic, EMA weights, one GH200. Δ is token − bits, paired over problems.

Step	Token head: seeds → mean	Bits head: seeds → mean	Δ token − bits, pooled [95% CI]	Ratio
100k	1.7 / 1.4 / 1.7 → **1.62%**	3.0 / 2.7 / 3.3 → **2.98%**	**-1.36 pp** [-2.05, -0.68]	0.54×
200k	3.9 / 4.3 / 3.9 → **4.07%**	7.9 / 7.2 / 6.5 → **7.20%**	**-3.13 pp** [-4.14, -2.12]	0.56×
300k	not evaluated (jobs cancelled at the stop)	9.7 / 9.4 / 9.7 → 9.60%	—	—
400k	not evaluated (jobs cancelled at the stop)	11.9 / 11.8 / 11.5 → 11.73%	—	—
500k (CSD3, batch 128)	2.2 / 2.6 / 2.9 → **2.55%**	5.4 / 4.5 / 4.9 → **4.93%**	**-2.38 pp** [-3.21, -1.57]	0.52×



Mean of 3 sampling seeds; the band spans the seeds. One training seed per arm. Source: results/tokens/summary_gh200.json.

Why it is worse: the programs do not run

At 200k steps	Token head	Bits head
Programs that execute at all	7.1%	23.0%
Correct, among executing programs (all problems)	57%	31%
Same, on the 160 problems where both arms execute	56%	41%
Typical failure (programs short enough to inspect)	NameError: a variable is defined under one name and used under a corrupted one (`meters_per_sprint` → `meters_per_printprint`)	stops early, before `return`

So the token head's deficit is in **writing code that runs**, not in the arithmetic. The raw "57% vs 31% correct when it runs" is partly a selection effect (token programs only run on short, easy problems), but on matched problems the token head is still +15 pp [+8, +23] more often right. Its failures look like losing track of identifiers across the program, a long-range consistency problem that per-bit prediction does not show as strongly.

Voting over the 3 samples among programs that run helps both arms (token 4.1% → 8.5%, bits 7.2% → 12.7%) and does not change the ranking.

4. Verdict and how much to trust it

> **Verdict: worse, not inconclusive**, for this head, this loss and this budget. Two independent matched pairs (Isambard batch 512 at 100k and 200k; CSD3 batch 128 at 500k) agree on a large, significant gap of the same size (~0.5–0.55×). What is **not** settled is whether a token head *could* win with a different loss, a wider head or a longer run.

- **One training seed per arm.** The 3 seeds are sampling seeds; the intervals cover problem and sampling noise, not training-run variance.
- **Only 100k and 200k are compared on Isambard.** Token 300k/400k and bits 500k were never evaluated, and the token run stopped at 414k.
- **Head and loss change together** (token cross-entropy vs bit score loss), and the token logits pass through a 128-wide head layer.
- **Training bugs found afterwards** (fixed 1 Oct, not re-run): GPUs other than rank 0 trained with σ_{data} 0.5 instead of the estimate (T1); every GPU drew the same noise (T3), which costs the 8-GPU

token run more than the 4-GPU bits run (≈64 vs ≈128 distinct noise draws per step). Neither is likely to explain a 2× accuracy gap, but neither is ruled out.

- **The learning rate (1e-4) is untuned for both heads.** The production bits model used 3e-4, which diverged in every from-scratch attempt here, so it is not a matched reference.

5. What to keep and what to do next

> **Keep the bits head** as the default. **Keep the token-head code path** (it works, is stable, and is the right tool for later tests) and the finding that its weakness is code validity, not arithmetic. **Do not** spend more training budget on this exact recipe.

If the question is reopened (GPU estimates; nothing has been run):

- Evaluate the existing token 300k/400k and bits 500k checkpoints (~4 GPU-h): does the ratio hold or the gap close?
- Separate head from loss: a token head with the score loss, or a bits head with cross-entropy; and try the wide `token_full` head.
- Confirm the T1/T3 fixes first (12k-step test, ~20 GPU-h for 3 seeds), then a second training seed for any re-run.

Sources. configs/tasks/tinygsm_tokens.py; scripts/hpc/arch/logits_gh200.sh; diffusion/continuous/losses.py, logit_postprocess.py; runs/tasks/tinygsm/{tok_gh500k_token_ce_s42, ord_gh500k_none_s42}/config.json, sigma_data.json, training_logs; logs/arch/cobit_gh_tok_*.log; results/tokens/summary_gh200.json, results/tokens/eval/*.json; results/cpu_reanalysis/t3_taxonomy.json, t4_voting.json, t5_selection.json; docs/experiments_ordering_objective.md (token sections); docs/cpu_reanalysis.md; CODEBASE.md (T1–T4, R4).